

Documentação Explicativa

Este documento descreve cada etapa do projeto de disciplina. O projeto se baseia na análise de artigos da BBC, utilizando técnicas de Processamento de Linguagem Natural (PLN) e aprendizado de máquina para categorizar notícias em cinco categorias: **business**, **entertainment**, **politics**, **sport**, **tech**.

Link do projeto no github: https://github.com/LMRocha/NLP_BBC-News-Classification

1. Visão Geral

- **Objetivo:** Classificar artigos da BBC e realizar buscas por similaridade.
 - **Dataset:** 2225 artigos da BBC, cada um com o respectivo corpus e categoria.
 - **Abordagem:** Pré-processamento textual, vetorização (TF-IDF, BoW, n-grams, Word2Vec), busca por similaridade de cosseno e treino de classificadores (SVM, Regressão Logística, Random Forest).
-

2. Importação de Bibliotecas

As bibliotecas são carregadas para:

- **Manipulação de dados:** `pandas`, `numpy`
- **Visualização:** `matplotlib`, `seaborn`, `wordcloud`
- **Processamento de texto:** `re`, `nltk`, `spacy`
- **Vetorização:** `sklearn.feature_extraction.text`, `gensim`
- **Modelos e métricas:** `sklearn.svm`, `sklearn.linear_model`, `sklearn.ensemble`, `sklearn.metrics`
- **Download do dataset:** `kaggle.api`

As stopwords em inglês e o lematizador do NLTK são baixados (`nltk.download`).

3. Download e Preparação dos Dados

O dataset é obtido via **Kaggle API** a partir do repositório `hgulitekin/bbcnewsarchive`. Os arquivos são descompactados no diretório `data/`. Em seguida, o arquivo `bbc-news-data.csv` é lido com `pandas`.

4. Funções de Preparação de Texto

Foram definidas funções para cada etapa, conforme a tabulação apresentada abaixo:

Função	Descrição
<code>tokenize(text)</code>	Divide o texto em palavras (split por espaços).
<code>normalize(text)</code>	Converte todo o texto para minúsculas.
<code>clean_text(text)</code>	Remove caracteres não alfabéticos (exceto espaços).
<code>lemmatize(tokens)</code>	Aplica lematização usando <code>WordNetLemmatizer</code> .
<code>stemmize(tokens)</code>	Aplica stemming com <code>PorterStemmer</code> .
<code>apply_pos_tagging(text)</code>	Usa spaCy para manter apenas substantivos, verbos e adjetivos (lematizados), removendo stopwords.
<code>preprocess_text(...)</code>	Função unificada que aplica normalização, limpeza, remoção de stopwords e opcionalmente lematização, stemização ou POS tagging.
<code>plot_word_cloud(...)</code>	Gera nuvem de palavras a partir dos textos processados.
<code>plot_bar_frequencia(...)</code>	Plota as 10 palavras mais frequentes em um gráfico de barras.

Após definir essas funções, os textos são pré-processados com lematização e stemização separadamente, gerando duas listas: `processed_text_lemmatize` e `processed_text_stemmize`. O resultado é armazenado em `df_clean`.

5. Funções de Vetorização

Quatro formas de representação vetorial são implementadas:

Função	Método	Parâmetros
<code>vetorizer_tfidf(docs)</code>	TF-IDF	<code>TfidfVectorizer()</code>
<code>vetorizer_bow(docs)</code>	Bag of Words	<code>CountVectorizer()</code>
<code>vetorizer_bow_ngrams(docs, range1, range2)</code>	BoW com n-grams	<code>ngram_range=(2, 2)</code>
<code>vetorizer_word2vec(sentences, vector_size=5, window=2, min_count=1)</code>	Word2Vec	Vetores de tamanho 5, janela 2, frequência mínima 1. O vetor de cada documento é a média dos vetores das palavras que estão no vocabulário.

Após a definição, os vetores são calculados para os textos lematizados e armazenados em um dicionário X.

6. Funções de Busca por Similaridade

Duas funções principais realizam consultas:

- **buscar(query, vectorizer, X_matrix, y_labels, original_texts, top_k=5)**
Aplica o mesmo pré-processamento à consulta, transforma-a com o **vectorizer** (TF-IDF, BoW ou n-grams) e calcula a similaridade de cosseno entre o vetor da consulta e cada documento. Retorna os **top_k** documentos mais similares, suas categorias e trechos.
- **buscar_w2v(query, w2v_model, doc_vectors, y_labels, original_texts, top_k=5)**
Similar, mas para Word2Vec: calcula o vetor da consulta como a média dos vetores das palavras e usa similaridade de cosseno nos vetores dos documentos.

Além disso, funções auxiliares:

- **plot_category_distribution(...)**: gráfico de pizza com a distribuição das categorias nos resultados.
- **plot_pca_projection(...)**: projeção PCA dos vetores dos documentos e da consulta, destacando os **top_k** resultados.

Exemplos de busca executados no notebook:

- "stock market crash" com TF-IDF
- "movie award" com BoW e com n-grams
- "computer" com Word2Vec (similaridades zero devido à pequena dimensão dos vetores e vocabulário limitado)

7. Funções de Análise Exploratória (EDA)

Função	Descrição
<code>count_tokens(text)</code>	Retorna o número de palavras no texto.
<code>get_top_words(category, df_clean, n=10)</code>	Retorna as 10 palavras mais frequentes em uma categoria.
<code>plot_analise_robustes(df_clean)</code>	Gera histograma da distribuição do número de palavras por documento e boxplot por categoria; também plota a contagem de documentos por categoria (desbalanceamento).
<code>plot_analise_top_word(df_clean)</code>	Exibe as 10 palavras mais frequentes por categoria e nuvens de palavras separadas por categoria.
<code>plot_word_cooccurrence_graph(...)</code>	Constrói um grafo de coocorrência (nós = palavras, arestas = pares que coocorrem no mesmo documento). Permite filtrar por categoria.

Essas funções são executadas para entender a distribuição dos dados, as palavras-chave de cada tópico e as relações semânticas entre termos.

8. Funções de Treino e Validação

- **treino_e_validacao(models, X_train, y_train, X_test=None, y_test=None, cv=None)**

Treina uma lista de modelos (ex.: `[('SVM', SVC()), ...]`). Se `X_test` e `y_test` são fornecidos, avalia acurácia, relatório de classificação e matriz de confusão. Retorna um dicionário com os resultados.

- **plot_validacao_modelos(metrics_df, plot_accuracy=True, plot_time=True, figsize=(10,5))**

Recebe um DataFrame com colunas "Modelo", "Acurácia", "Tempo (s)" e plota gráficos de barras comparativos.

9. Pipeline de Execução

O notebook executa as seguintes etapas (células finais):

1. **Preparação textual** – Geração dos textos lematizados e stemizados.
 2. **Visualização inicial** – Nuvem de palavras e gráfico de frequência dos textos lematizados.
 3. **Vetorização** – Criação dos vetores TF-IDF, BoW, n-grams e Word2Vec.
 4. **Busca por similaridade** – Demonstração com consultas e projeção PCA.
 5. **Análise EDA** – Execução das funções de análise (tamanho dos documentos, palavras-chave, coocorrência).
 6. **Treino e validação** – (Nota: no notebook fornecido os modelos não são treinados explicitamente, mas as funções estão definidas; a seção final indica que seriam utilizadas.)
-

Observações Importantes

- **Pré-processamento:** Foram aplicadas lematização e stemização separadamente. A lematização foi a mais utilizada nas etapas posteriores.
- **Word2Vec:** Com `vector_size=5` e `min_count=1`, os vetores são de baixa dimensão e qualquer palavra é considerada. Isso pode explicar as similaridades nulas na busca "computer".
- **Desbalanceamento:** A categoria `sport` possui 511 artigos, enquanto `entertainment` tem 386 – isso pode afetar a performance dos classificadores.
- **Palavras-chave:** Por categoria, as palavras mais frequentes refletem os tópicos (ex.: `said`, `mr` em `business`; `film`, `award` em `entertainment`; `game`, `player` em `sport`; `technology`, `mobile` em `tech`).

- **Grafo de coocorrência:** Útil para visualizar associações semânticas; o notebook gera gráficos para todos os documentos e para categorias específicas.
-

Análise da execução das etapas

Etapa 1 – Preparação textual

Foi realizado todo o processo de normalização, tokenização e purificação dos documentos (*remoção de dados numéricos, stopwords e acentuações*).

Posteriormente foi aplicado as estratégias de *lemmatizing*, *stemmizing* e *POS*. A estratégia selecionada foi *lemmatizing* pois a técnica de *stemmizing* “cortava” em excesso o contexto das palavras podendo gerar falhas na interpretação nos treinamentos do modelo. Não houve a aplicação do *POS* devido a lentidão de sua aplicação ao dataset (*aguardo de aproximadamente 15 minutos sem finalização*).

Etapa 2 – Vetorização

Busca por similaridade

Foram utilizados as técnicas de *TF-IDF*, *BoW*, *Bow N-gram* e *Word2Vec* para a transformação dos documentos em vetores. Foi gerado um dicionário para armazenar os vetorizados, processados com *lemmatizing* cada qual com a sua estratégia.

Para cada modelo de vetorização foi realizada uma implementação uma busca por similaridade.

Analisando os resultados foi verificado que a busca com os melhores índices foi de longe o *Word2Vec*.

Análise EDA textual

- Realizada a contagem de tokens por categoria.
- Listagem das palavras mais frequentes
- Visualização do balanceamento por categorias do dataset
 - Dataset minimamente desbalanceado, não o suficiente para impactar nos resultados finais.
- Realizada a análise de grafo de coocorrência com 25 nós, utilizando a categoria “tech”.
 - Foi verificada uma densidade de 1000 o que indica um grafo denso na proximidade de suas palavras.
 - Arestas grossas representam forte associação entre palavras, por exemplo *mobile-phone*.

Etapa 3 – Treinamento e métricas

O treinamento e o teste foram realizados através do aprendizado supervisionado, utilizando os modelos de classificação *SVM*, *Logistic Regression* e *Random Forest*. De forma a realizar uma análise comparativa entre os diferentes tipos de vetorização e a sua eficiência frente aos modelos utilizados, foi iterado para cada conjunto de textos processados o treinamento para os três modelos.

Analisando o resultado final podemos concluir que ambas as métricas foram muito boas, sendo a única técnica de vetorização com as menores métricas a *Word2Vec*.

Foi analisado também o tempo de treinamento do modelo mediante a técnica de vetorização utilizada, e concluímos que a melhor abordagem é BoW-Ngra → Logistic Regression, que apresentou o menor tempo de treinamento e o maior percentual de métricas.

Referente as métricas abaixo do geral do *Word2Vec*, a principal argumentação está em sua performance reduzida para lidar com algoritmos de classificação, devido a densidade de seus vetores para as palavras. Entra em pauta, também, que apesar de mais de 2 mil documentos, ainda é uma quantidade relativamente pequena para o uso performativo do *Word2Vec*.

Conclusão

Este notebook fornece uma base sólida para classificação de textos em inglês, cobrindo desde o pré-processamento até a avaliação de modelos. As funções são modulares e podem ser reutilizadas em outros projetos de PLN. A análise exploratória permite compreender a natureza dos dados, e os mecanismos de busca demonstram a utilidade da representação vetorial para recuperação de informação.